HW 5 Solutions

ENGR 21, Fall 2026.

October 8, 2026

Solutions

1 Stacking and looping

1.1 When stacking works

Write a function that takes as input two `numpy` arrays (assumed to be 2-dimensional — if not, it should print an error message and exit.

Your function should return a tuple of Booleans indicating:

1. Whether the two arrays can be `hstack`'ed
2. Whether the two arrays can be `vstack`'ed

```
def check_stacking(array1, array2):
  # check if array1 and array2 can be hstacked. Store result in can_hstack
  can_hstack = False
  # check if array1 and array2 can be vstacked. Store result in can_vstack
  can_vstack = False
  return (can_hstack, can_vstack)
```

Solution

```
def check_stacking(array1, array2):
    if np.shape(array1)[0] == np.shape(array2)[0]:
        can_hstack = True
    else:
        can_hstack = False
    if np.shape(array1)[1] == np.shape(array2)[1]:
        can_vstack = True
    else:
        can_vstack = False
    return (can_hstack, can_vstack)
```

1.2 Looping over a numpy array

Consider the `numpy` array shown below, which has two dimensions.

```
array([[14., 15., 30.],
       [ 3., 16.,  4.],
       [87., 35., 47.],
       [96., 37., 85.]])
```

Write two functions in Python that take as argument a 2-dimensional `numpy` array and print every element inside the array in two different ways:

1. In row-first-then-column order, starting from the `[0,0]` position, i.e., for the above example, 14 — 15 — 30 — 3 — 16 — … — 37 — 85
2. In coluumn-first-then-row order, starting from the `[-1,0]` position. i.e., for the above example, 96 — 87 — 3 — 14 — 37 — 35 — … — 4 — 30

Your functions should work on 2-dimensional arrays of arbitrary shape.

You need two separate functions here.

Solution

```
def print1(a):
 for j in range(len(a[:,0])):
    for i in range(len(a[0,:])):
        print(a[j,i])
```

```
def print2(a):
 for i in range(len(a[0,:])):
    for j in range(-1,-len(a[:,0])-1,-1):
        print(a[j,i])
```

2 Indexing and sorting etc.

The following Python variable gives 70 student numbers and 70 scores for each of those students in one 70 x 2 numpy array. Look online for the array.

2.1 Calculate the mean, median, maximum, and minimum score

Write a Python program that prints out the mean, median, maximum and minimum score values, using print statements such as the following.

```
print(f"The mean score is {x} points")
```

Solution

Assuming that the given array has been assigned to a variable called `student_score`, the following lines solve this problem.

```
print(f"The mean score is {np.mean(student_score[:,1]):.2f} points")
print(f"The median score is {np.median(student_score[:,1]):.2f} points")
print(f"The maximum score is {np.max(student_score[:,1]):.2f} points")
print(f"The minimum score is {np.min(student_score[:,1]):.2f} points")
```

2.2 An array with students whose score are above 87.

Write a Python program that prints to the screen an Nx2 array consisting of only those students (including student number + that student's score) whose score was greater than 87.

Solution

Assuming that the given array has been assigned to a variable called `student_score`, the single line of code

```
print(student_score[student_score[:,1]>87,:])
```

solves this problem.

2.3 A sorted array of student numbers and scores

For this problem, you will need to make use of the function `argsort`.

Print a 2-dimensional array of student numbers and scores where the scores have been sorted from highest to lowest. Your sorting must preserve which score belongs to which student, i.e., both of your columns need to be re-arranged. For example, student 3 must still have 86.3 points.

Solution

The following one-liner solves this problem, assuming that the given array has been assigned to a variable called `student_score`.

```
print(student_score[np.argsort(student_score[:,1]),:])
```

To make the solution more transparent, however, we can break it down.

```
# Extract scores and students
scores = student_score[:,1]
students = student_score[:,0]

# Use `argsort` to find the indices that, if applied
# to `scores`, would re-arrange them into the correct
# order
indices_to_reorder = np.argsort(scores)

# Use these seventy indices to access both columns
print(student_score[indices_to_reorder,:])
```

2.4 Logical Indexing with multiple conditions

Print the student number and score of all students whose:

- student number is an even number, **and**
- score (before the decimal point) is an even number

> **Note**
>
> - the function `np.int64()` converts floating-point numbers to integers
> - Use `np.set_printoptions(precision=2)`

3 Manipulating Arrays

3.1 Reshape

Write code that `reshape`s the given matrix into eight different shapes if possible. Each of your answers should result in a different shape.

```
>>> y = np.random.random((3,5,6))
```

```
array([[[9.22e-01, 7.75e-01, 8.75e-01, 5.01e-01, 9.87e-01, 7.30e-01],
        [3.95e-01, 4.52e-01, 6.85e-01, 3.12e-01, 5.58e-01, 2.00e-01],
        [4.04e-01, 3.46e-01, 3.41e-01, 1.24e-01, 9.56e-01, 7.27e-01],
        [2.16e-01, 8.41e-01, 2.12e-01, 8.09e-01, 9.91e-01, 8.28e-01],
        [8.99e-01, 6.94e-01, 1.37e-01, 2.06e-01, 8.51e-01, 9.95e-01]],

       [[2.00e-01, 1.34e-01, 4.01e-01, 2.34e-01, 8.29e-01, 5.76e-01],
        [4.20e-02, 2.89e-02, 5.70e-01, 1.04e-01, 7.85e-01, 8.80e-01],
        [1.45e-01, 7.66e-01, 5.24e-01, 7.34e-01, 4.43e-01, 7.75e-01],
        [3.47e-01, 9.07e-01, 4.92e-02, 1.12e-01, 3.68e-02, 4.53e-01],
        [5.28e-01, 1.69e-04, 8.48e-01, 4.62e-01, 4.59e-02, 7.80e-01]],
```

```
       [[6.35e-01, 2.82e-01, 3.60e-01, 1.35e-01, 5.22e-01, 6.88e-01],
        [8.87e-01, 7.33e-01, 8.14e-01, 6.00e-01, 1.03e-01, 3.29e-01],
        [1.38e-01, 7.47e-01, 7.43e-03, 8.72e-01, 6.16e-01, 7.64e-01],
        [1.94e-01, 2.36e-01, 4.48e-02, 6.36e-01, 3.62e-01, 2.51e-01],
        [1.20e-01, 5.97e-02, 9.59e-01, 7.50e-01, 9.64e-01, 9.39e-02]]])
```

Solution

The following should work to reshape the given array.

1. `y.reshape((15,6))`
2. `y.reshape((5,18))`
3. `y.reshape((30,3))`
4. `y.reshape((6,15))`
5. `y.reshape((18,5))`
6. `y.reshape((3,30))`
7. `y.reshape((6,5,3))`
8. `y.reshape((45,2))`

3.2 3D slicing

1. What one line of code will gather all red-highlighted values into one array?
2. What one line of code will gather the orange-highlighted values into one place in the following form:

```
array([[0.31, 0.56],
       [0.2 , 0.1 ],
       [0.79, 0.88]])
```

Solution

1. `new1 = y[:,1,1]`
2. `new2 = np.reshape(y[:2,1,3:],(3,2))`

4 The Farmer Was Replaced

Pumpkins in TFWR show a special kind of behavior — they can get bigger if they are planted next to each other in patterns such as 2x2, 3x3, 4x4, etc, but they are also subject to the vagaries of fate and can spontaneously turn out rotten.

Tip

Download a save file for the game here.

Write a program for your drone that plants pumpkins in a formation **at least** 4x4 large in such a way that it **waits until a big pumpkin appears** before it harvests healthy pumpkins. The accompanying video shows you only one way to do this; there are lots of ways to accomplish this correctly.

Note

In the video, the drone moves past pumpkins after planting them. Another approach could be to wait and see if a pumpkin will turn out rotten, and to only go ahead if the pumpkin has been determined to not be rotten.

Your code should run continuously, planting according to a pre-set pattern inside a `while` loop and harvesting everything it plants. Ideally, your farm should be self-sustaining, i.e., you should plant a combination of crops that allows you to continue harvesting for ever (yay sustainability).

It is recommended that you use dictionaries to determine what crops to plant where. For example, the following dictionary and list-of-list of plants was used here:

```
# An example pattern
pattern3 = [["t", "g", "g", "g","g","c","t","c"],
            ["g", "p", "p", "p","p","c","t","c"],
            ["g", "p", "p", "p","p","c","g","c"],
            ["g", "p", "p", "p","p","c","g","c"],
            ["g", "p", "p", "p","p","c","t","c"],
            ["c", "c", "c", "c","c","c","g","t"],
            ["t", "t", "g", "g","g","t","c","c"],
            ["c", "c", "c", "c","c","g","c","c"]]

# The dictionary definition
crops = {
  "p": Entities.Pumpkin,
  "t": Entities.Tree,
```

```
  "g": Entities.Grass,
  "c": Entities.Carrot
}
```

Solution

```
# An example pattern
pattern3 = [["t", "g", "g", "g","g","c","t","c"],
      ["g", "p", "p", "p","p","c","t","c"],
      ["g", "p", "p", "p","p","c","g","c"],
      ["g", "p", "p", "p","p","c","g","c"],
      ["g", "p", "p", "p","p","c","t","c"],
      ["c", "c", "c", "c","c","c","g","t"],
      ["t", "t", "g", "g","g","t","c","c"],
      ["c", "c", "c", "c","c","g","c","c"]]

pumpkinsize = 16

# The dictionary definition
crops = {
  "p": Entities.Pumpkin,
  "t": Entities.Tree,
  "g": Entities.Grass,
  "c": Entities.Carrot
}

def zeroloc():
  xdist = get_pos_x()
  ydist = get_pos_y()
  for i in range(xdist):
    move(West)
  for i in range(ydist):
    move(South)
  return None

# Your function definition
def plant_pattern(pat):
  for i in range(get_world_size()):
    for j in range(get_world_size()):
      current = get_entity_type()
      if can_harvest():
        harvest()
      plant(crops[pattern3[i][j]])
      move(East)
    move(North)

# Initialize and till
clear()
for row in range(get_world_size()):
  for col in range(get_world_size()):
    if get_ground_type() == Grounds.Grassland:
      harvest()
      till()
    move(East)
  move(North)


while True:
```